

Import library

```
1 import numpy as np
2 import pandas as pd
3 import yfinance as yf
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 import warnings
7 warnings.filterwarnings('ignore')
```

DATA COLLECTION

```
# This is formatted as code
```

```
1 # step 1: DATA COLLECTION Stock Market Prediction - Nifty 50 till date
2 ticker = '^NSEI'
3 data = yf.download(ticker, start='2019-01-01', end='2026-05-01')
4 print(data.isnull().sum())
5 print(data.duplicated().sum())
6 data.reset_index(inplace=True)
7 print(f'Dataset Shape: {data.shape}')
8 print(data.head())
9 print(data.tail())
10 print(data.info())
11 print(data.describe())
```

```
Price    Volume
Ticker   ^NSEI
0        309700
1        286200
2        296600
3        269400
4        277700
Price    Date      Close      High      Low      Open \
Ticker   ^NSEI      ^NSEI      ^NSEI      ^NSEI
1801  2026-04-24  23897.949219  24206.000000  23813.650391  24100.550781
1802  2026-04-27  24092.699219  24130.699219  23936.199219  23945.449219
1803  2026-04-28  23995.699219  24181.800781  23957.050781  24049.900391
1804  2026-04-29  24177.650391  24334.699219  24059.949219  24096.900391
1805  2026-04-30  23997.550781  24087.449219  23796.849609  23996.949219

Price    Volume
Ticker   ^NSEI
1801     438400
1802     430300
1803     554100
1804     531100
1805     505500
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1806 entries, 0 to 1805
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
75%
```

```

75%      2024-07-02 18:00:00  22820.350195  22923.887095
max      2026-04-30 00:00:00  26328.550781  26373.199219
std      NaN                5121.508270   5132.383054

Price      Low      Open      Volume
Ticker     ^NSEI     ^NSEI     ^NSEI
count    1806.000000  1806.000000  1.806000e+03
mean     17773.248114  17881.428713  3.916383e+05
min       7511.100098  7735.149902  0.000000e+00
25%      12291.524902  12336.599609  2.587500e+05
50%      17582.774414  17701.475586  3.282000e+05
75%      22637.999512  22807.075195  4.932750e+05
max      26210.050781  26333.699219  1.811000e+06
std       5110.488636   5119.611178  1.957126e+05

```

DATA PREPROCESSING

```

1 import pandas as pd
2
3 # Flatten MultiIndex-like columns if they exist
4 # The columns are likely tuples like ('Date', ) or ('Close', '^NSEI')
5 # The 'ticker' variable is available from the kernel state, which is '^NSEI'.
6 if any(isinstance(col, tuple) for col in data.columns):
7     new_columns = []
8     for col in data.columns:
9         if isinstance(col, tuple):
10             # Handle ('Date', ) as 'Date'
11             if len(col) == 2 and col[0] == 'Date' and col[1] == '':
12                 new_columns.append(col[0])
13             # Handle ('Close', '^NSEI') as 'Close' and similar columns
14             elif len(col) == 2 and col[1] == ticker:
15                 new_columns.append(col[0])
16             # Fallback for other tuple columns
17             else:
18                 new_columns.append('.'.join(map(str, col)))
19         else:
20             new_columns.append(col)
21     data.columns = new_columns
22
23 data.dropna(inplace=True)
24 data.drop_duplicates(inplace=True)
25
26 rows_before_outlier_removal = data.shape[0]
27
28 Q1 = data['Close'].quantile(0.25)
29 Q3 = data['Close'].quantile(0.75)
30 IQR = Q3 - Q1
31 lower_limit = Q1 - 1.5 * IQR
32 upper_limit = Q3 + 1.5 * IQR
33 data = data[(data['Close'] >= lower_limit) & (data['Close'] <= upper_limit)]
34
35 print(f"Shape of DataFrame after outlier removal: {data.shape}")
36
37 print(f"Removed {rows_before_outlier_removal - data.shape[0]} rows containing Close price outliers."

```

```

Shape of DataFrame after outlier removal: (1806, 6)
Removed 0 rows containing Close price outliers.

```

✓ Creating Lag Features for MACD,RSI, BB_Upper & BB_Lower

```

1 def add_technical_indicators(data):
2     # Simple Moving Averages
3     data['SMA_10'] = data['Close'].rolling(window=10).mean()
4     data['SMA_20'] = data['Close'].rolling(window=20).mean()
5     data['SMA_50'] = data['Close'].rolling(window=50).mean()
6
7     # Exponential Moving Averages
8     data['EMA_12'] = data['Close'].ewm(span=12, adjust=False).mean()
9     data['EMA_26'] = data['Close'].ewm(span=26, adjust=False).mean()

```

```

10
11 # MACD
12 data['MACD'] = data['EMA_12'] - data['EMA_26']
13 data['Signal'] = data['MACD'].ewm(span=9, adjust=False).mean()
14
15 # RSI (14-day)
16 delta = data['Close'].diff()
17 gain = delta.where(delta > 0, 0).rolling(14).mean()
18 loss = (-delta.where(delta < 0, 0)).rolling(14).mean()
19 rs = gain / loss
20 data['RSI'] = 100 - (100 / (1 + rs))
21 # Bollinger Bands
22 data['BB_Mid'] = data['Close'].rolling(20).mean()
23 data['BB_Std'] = data['Close'].rolling(20).std()
24 data['BB_Upper'] = data['BB_Mid'] + 2 * data['BB_Std']
25 data['BB_Lower'] = data['BB_Mid'] - 2 * data['BB_Std']
26
27 # Lag features
28 data['MA10'] = data['Close'].rolling(10).mean()
29 data['MA50'] = data['Close'].rolling(50).mean()
30 data['Lag1'] = data['Close'].shift(1)
31 data['Lag2'] = data['Close'].shift(2)
32 data['Return'] = data['Close'].pct_change()
33 data["lag1"] = data["Close"].shift(1)
34 data["std5"] = data["Close"].rolling(5).std()
35
36 # Average True Range
37 data['TR'] = np.maximum(data['High'] - data['Low'],
38                          np.maximum(abs(data['High'] - data['Close'].shift(1)),
39                                     abs(data['Low'] - data['Close'].shift(1))))
40 data['ATR'] = data['TR'].rolling(14).mean()
41
42 # On-Balance Volume
43 data['OBV'] = (np.sign(data['Close'].diff()) * data['Volume']).fillna(0).cumsum()
44
45 # Daily Return
46 data['Daily_Return'] = data['Close'].pct_change()
47
48 return data.dropna()
49
50 data = add_technical_indicators(data)
51 print(f'Shape after feature engineering: {data.shape}')
52 print(data[['Date', 'Close', 'SMA_20', 'RSI', 'MACD', 'BB_Upper', 'BB_Lower', 'MA10', 'MA50', 'Lag1', 'Lag2']])

```

Shape after feature engineering: (1757, 29)

	Date	Close	SMA_20	RSI	MACD \
1801	2026-04-24	23897.949219	23634.604980	67.064645	91.217849
1802	2026-04-27	24092.699219	23693.619922	66.481484	88.858143
1803	2026-04-28	23995.699219	23728.082422	63.004592	78.258850
1804	2026-04-29	24177.650391	23795.984961	53.387710	83.577346
1805	2026-04-30	23997.550781	23879.292480	54.246951	72.424890

	BB_Upper	BB_Lower	MA10	MA50	Lag1 \
1801	25037.443859	22231.766102	24206.540039	24283.630039	24173.050781
1802	25067.543306	22319.696538	24210.750000	24248.138008	23897.949219
1803	25095.677963	22360.486880	24226.054883	24209.348984	24092.699219
1804	25107.357378	22484.612544	24220.689844	24173.825000	23995.699219
1805	24996.184940	22762.400021	24200.769922	24137.632031	24177.650391

	Lag2
1801	24378.099609
1802	24173.050781
1803	23897.949219
1804	24092.699219
1805	23995.699219

Weekly Candlestick Chart

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3

```

```
4 # Install mplfinance if not already installed
5 try:
6     import mplfinance as mpf
7 except ModuleNotFoundError:
8     !pip install mplfinance
9     import mplfinance as mpf
10
11 # Prepare data for plotting
12 # The original 'data' DataFrame needs to have 'Date' as index for resampling and mpf.plot
13 data_plot = data.copy()
14 data_plot['Date'] = pd.to_datetime(data_plot['Date'])
15 data_plot.set_index('Date', inplace=True)
16
17 # Resample to Weekly
18 data_weekly = data_plot.resample('W').agg({
19     'Open': 'first',
20     'High': 'max',
21     'Low': 'min',
22     'Close': 'last',
23     'Volume': 'sum'
24 }).dropna()
25
26 fig, axes = mpf.plot(data_weekly, type='candle', style='yahoo',
27                      title='Nifty 50 Weekly Candlestick Chart',
28                      ylabel='Price',
29                      volume=True, ylabel_lower='Volume',
30                      show_nontrading=False,
31                      figscale=1.5,
32                      figratio=(16,9),
33                      returnfig=True)
34
35 fig.suptitle('Nifty 50 Weekly Candlestick Chart', y=0.92)
36
37 for ax in axes:
38     if isinstance(ax, plt.Axes):
39         ax.yaxis.set_label_coords(-0.05, 0.5)
40
41 plt.tight_layout()
42 plt.show()
```

Collecting mplfinance

Downloading mplfinance-0.12.10b0-py3-none-any.whl.metadata (19 kB)

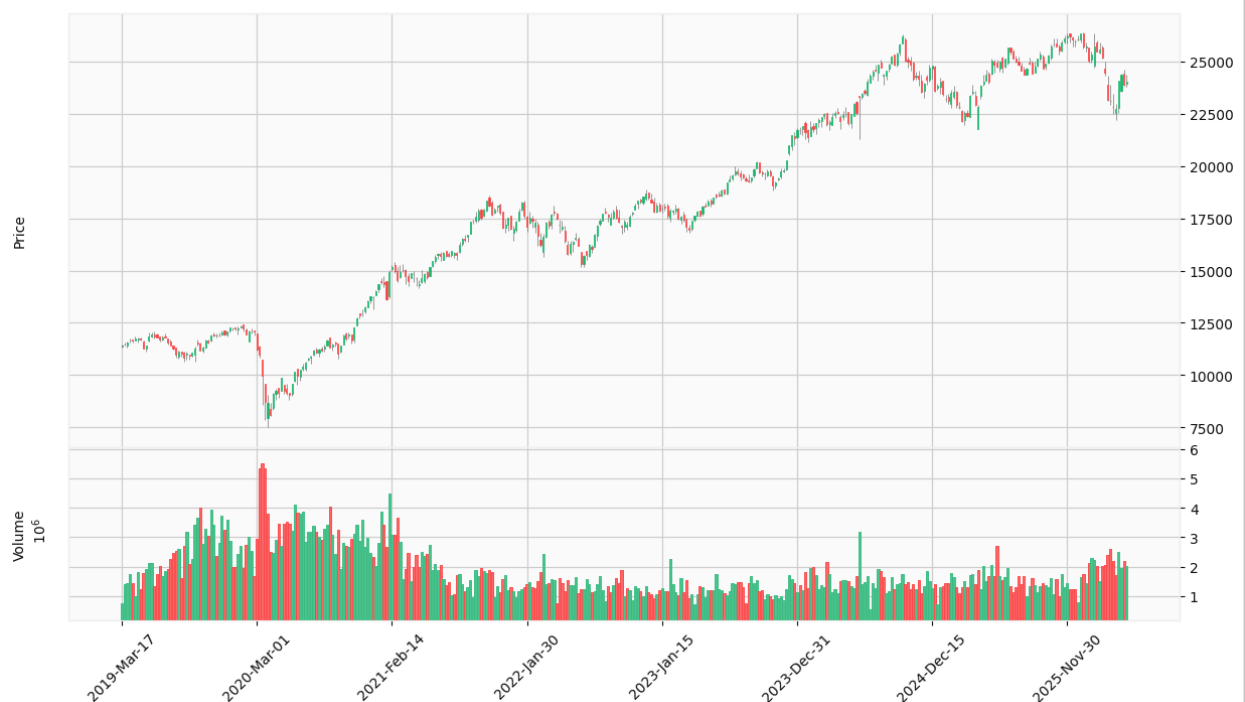
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from mplfinance) (3.8.0)
 Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from mplfinance) (2.2.2)
 Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.1.1)
 Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.53.0)
 Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.5)
 Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)
 Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (24.1)
 Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (10.4.0)
 Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.1.4)
 Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0)
 Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->mplfinance) (2024.2)
 Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->mplfinance) (2024.2)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil) (1.17.0)
 Downloading mplfinance-0.12.10b0-py3-none-any.whl (75 kB)

75.0/75.0 kB 3.8 MB/s eta 0:00:00

Installing collected packages: mplfinance

Successfully installed mplfinance-0.12.10b0

Nifty 50 Weekly Candlestick Chart



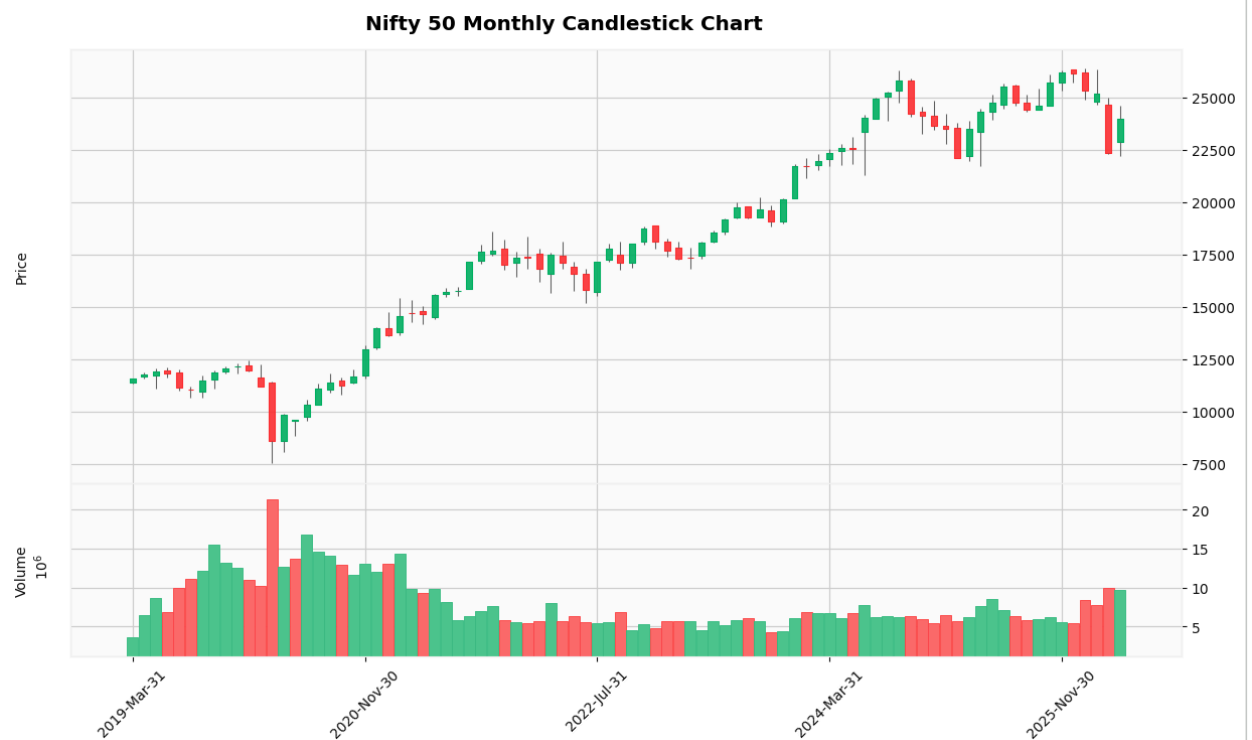
Monthly Candlestick Chart

```
1 # Prepare data for plotting
2 # The original 'data' DataFrame needs to have 'Date' as index for resampling and mpf.plot
3 df_plot = data.copy()
4 df_plot['Date'] = pd.to_datetime(df_plot['Date'])
5 df_plot.set_index('Date', inplace=True)
6
7 # Resample to Monthly
8 # Fix: Changed frequency from 'M' to 'ME' as per pandas 2.2.0+ requirements
9 df_monthly = df_plot.resample('ME').agg({
10     'Open': 'first',
11     'High': 'max',
12     'Low': 'min',
13     'Close': 'last',
14     'Volume': 'sum'
15 }).dropna()
16
17 fig, axes = mpf.plot(df_monthly, type='candle', style='yahoo',
18                     title='Nifty 50 Monthly Candlestick Chart',
19                     ylabel='Price',
```

```

20     volume=True, ylabel_lower='Volume',
21     show_nontrading=False,
22     figscale=1.5,
23     figratio=(16,9),
24     returnfig=True)
25
26 fig.suptitle('Nifty 50 Monthly Candlestick Chart', y=0.92)
27
28 for ax in axes:
29     if isinstance(ax, plt.Axes):
30         ax.yaxis.set_label_coords(-0.05, 0.5)
31
32 plt.tight_layout()
33 plt.show()

```



Quarterly Candlestick Chart

```

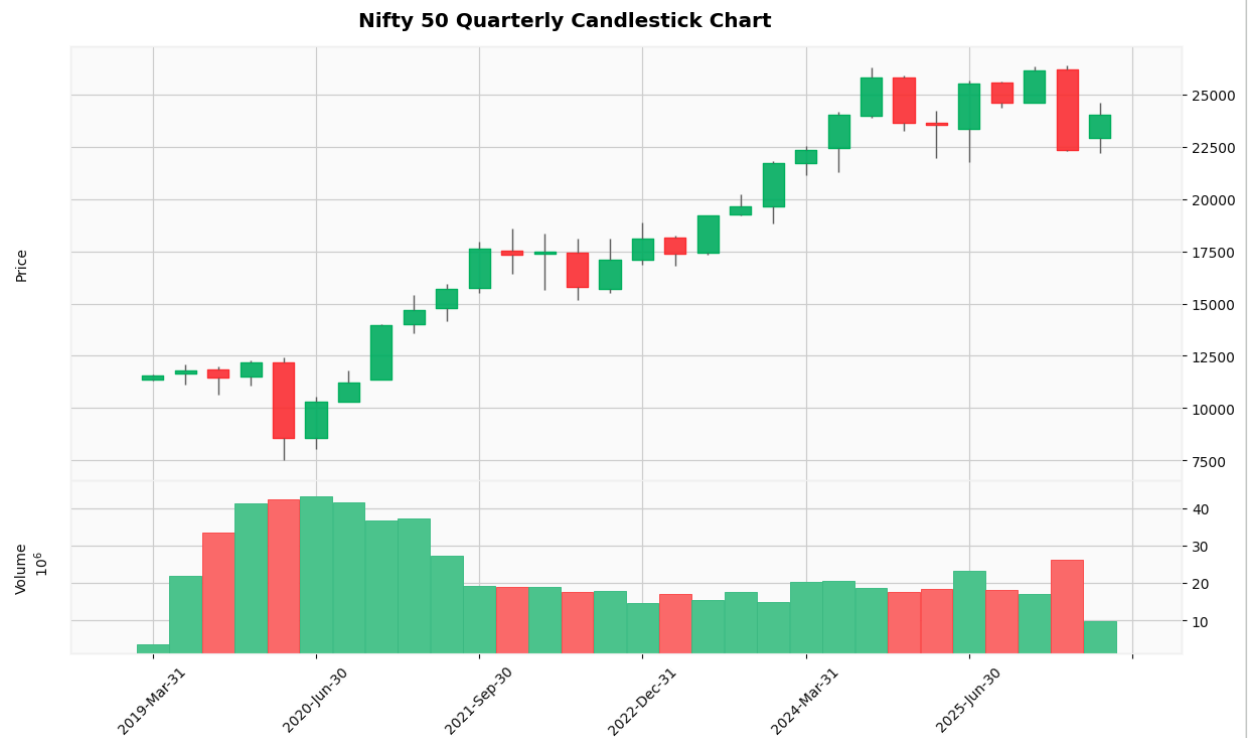
1 # Resample to Quarterly
2 # Fix: Changed frequency from 'Q' to 'QE' as per pandas 2.2.0+ requirements
3 df_quarterly = df_plot.resample('QE').agg({
4     'Open': 'first',
5     'High': 'max',
6     'Low': 'min',
7     'Close': 'last',
8     'Volume': 'sum'
9 }).dropna()
10
11 fig, axes = mpf.plot(df_quarterly, type='candle', style='yahoo',
12                     title='Nifty 50 Quarterly Candlestick Chart',
13                     ylabel='Price',
14                     volume=True, ylabel_lower='Volume',
15                     show_nontrading=False,
16                     figscale=1.5,
17                     figratio=(16,9),
18                     returnfig=True)
19
20 fig.suptitle('Nifty 50 Quarterly Candlestick Chart', y=0.92)
21
22 for ax in axes:
23     if isinstance(ax, plt.Axes):
24         ax.yaxis.set_label_coords(-0.05, 0.5)

```

```

25
26 plt.tight_layout()
27 plt.show()

```



✓ MACHINE LEARNING MODELS

```

=====
=====

```

```

1 from sklearn.linear_model import LinearRegression
2 from sklearn.ensemble import RandomForestRegressor
3 from sklearn.svm import SVR
4 from sklearn.preprocessing import MinMaxScaler
5 from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
6 from xgboost import XGBRegressor
7 import math
8
9 # -----
10 # 2.1 Prepare ML Features and Target
11 # -----
12 feature_cols = ['Open', 'High', 'Low', 'Volume', 'SMA_10', 'SMA_20', 'SMA_50',
13                 'EMA_12', 'EMA_26', 'MACD', 'Signal', 'RSI',
14                 'BB_Upper', 'BB_Lower', 'MA10', 'MA50', 'Lag1', 'Lag2', 'ATR', 'OBV', 'Daily_Return']
15
16 X = data[feature_cols].values
17 y = data['Close'].values
18
19 # Train / Test Split (80/20) - chronological
20 split = int(len(X) * 0.80)
21 X_train, X_test = X[:split], X[split:]
22 y_train, y_test = y[:split], y[split:]
23
24 # Scale features
25 scaler_X = MinMaxScaler()
26 X_train_sc = scaler_X.fit_transform(X_train)
27 X_test_sc = scaler_X.transform(X_test)

```

```

1 # 2.2 Evaluation Helper
2 # -----
3 def evaluate(name, y_true, y_pred):
4     rmse = math.sqrt(mean_squared_error(y_true, y_pred))
5     mae = mean_absolute_error(y_true, y_pred)
6     mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
7     r2 = r2_score(y_true, y_pred)
8     print(f'\n{name}')
9     print(f' RMSE : {rmse:.2f}')
10    print(f' MAE : {mae:.2f}')
11    print(f' MAPE : {mape:.2f}%')
12    print(f' R2 : {r2:.4f}')
13    return {'Model': name, 'RMSE': rmse, 'MAE': mae, 'MAPE': mape, 'R2': r2}
14
15 results = []

```

```

1 # 2.3 Linear Regression
2 # -----
3 lr = LinearRegression()
4 lr.fit(X_train_sc, y_train)
5 y_pred_lr = lr.predict(X_test_sc)
6 results.append(evaluate('Linear Regression', y_test, y_pred_lr))

```

```

Linear Regression
RMSE : 47.42
MAE : 36.26
MAPE : 0.15%
R2 : 0.9980

```

```

1 # -----
2 # 2.4 Random Forest Regressor
3 # -----
4 from sklearn.model_selection import GridSearchCV
5 from sklearn.ensemble import RandomForestRegressor
6 rf = RandomForestRegressor(n_estimators=200, max_depth=14,
7                             random_state=40, n_jobs=-1)
8 rf.fit(X_train_sc, y_train)
9 y_pred_rf = rf.predict(X_test_sc)
10 results.append(evaluate('Random Forest', y_test, y_pred_rf))
11

```

```

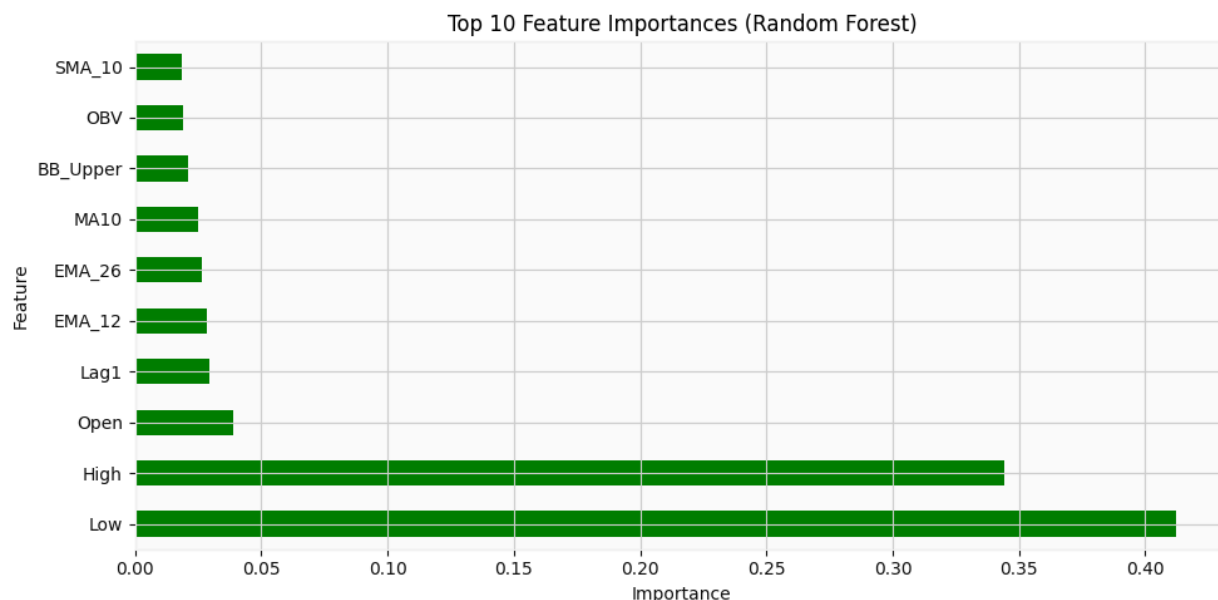
Random Forest
RMSE : 179.35
MAE : 129.83
MAPE : 0.54%
R2 : 0.9717

```

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Get feature importances from the Random Forest model
5 rf_feat_imp = pd.Series(rf.feature_importances_, index=feature_cols)
6
7 # Sort and plot the top 10 feature importances
8 rf_feat_imp.sort_values(ascending=False).head(10).plot(kind='barh', figsize=(10,5),
9     color='green', title='Top 10 Feature Importances (Random Forest)')
10 plt.xlabel('Importance')
11 plt.ylabel('Feature')
12 plt.tight_layout()
13 plt.show()

```



```

1 # -----
2 # 2.5 XGBoost Regressor
3 # -----
4 xgb = XGBRegressor(
5     n_estimators=500,
6     learning_rate=0.04,
7     max_depth=6,
8     min_child_weight=3,
9     subsample=0.8,
10    colsample_bytree=0.8,
11    gamma=0.2,
12    reg_alpha=0.2,
13    reg_lambda=2,
14    random_state=40
15 )
16 xgb.fit(X_train_sc, y_train,
17         eval_set=[(X_test_sc, y_test)], verbose=False)
18 y_pred_xgb = xgb.predict(X_test_sc)
19 results.append(evaluate('XGBoost', y_test, y_pred_xgb))

```

XGBoost

RMSE : 159.63
 MAE : 115.42
 MAPE : 0.47%
 R2 : 0.9776

```

1 # -----
2 # 2.6 Support Vector Regression
3 # -----
4 from sklearn.preprocessing import MinMaxScaler
5 svr = SVR(kernel='rbf', C=1500, gamma=0.01, epsilon=0.01)
6 svr.fit(X_train_sc, y_train)
7 y_pred_svr = svr.predict(X_test_sc)
8 scaler = MinMaxScaler()
9 X_scaled = scaler.fit_transform(X)
10 results.append(evaluate('SVR (RBF)', y_test, y_pred_svr))

```

SVR (RBF)

RMSE : 221.62
 MAE : 182.02
 MAPE : 0.74%
 R2 : 0.9568

```

1 # -----
2 # 2.7 Results Summary Table

```

```

3 # -----
4 results_df = pd.DataFrame(results)
5 print('\n===== ML MODEL COMPARISON =====')
6 print(results_df.sort_values('RMSE').to_string(index=False))
7

```

```

===== ML MODEL COMPARISON =====
      Model      RMSE      MAE      MAPE      R2
Linear Regression  47.420234  36.264360  0.149330  0.998022
      XGBoost 159.633679 115.416110  0.465841  0.977583
      Random Forest 179.352152 129.829345  0.535465  0.971703
      SVR (RBF) 221.621440 182.020730  0.738900  0.956794

```

✓ Deep Learning Models (LSTM, GRU, CNN-LSTM)

```

1 # =====
2 # BLOCK 3: DEEP LEARNING MODELS
3 # =====
4
5 import tensorflow as tf
6 from tensorflow.keras.models import Sequential
7 from tensorflow.keras.layers import (LSTM, GRU, Dense, Dropout,
8   Conv1D, MaxPooling1D, Flatten, BatchNormalization)
9 from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
10 from tensorflow.keras.optimizers import Adam
11

```

```

1 # -----
2 # 3.1 Prepare Sequential Data for DL (Lookback = 60 days) - now with multiple features
3 # -----
4 LOOKBACK = 60
5
6 # Scale all relevant features for DL
7 # We'll use the feature_cols we defined for ML models
8
9 # First, prepare the data with selected features
10 data_features = data[feature_cols].copy()
11
12 # Scale the features
13 scaler_features = MinMaxScaler(feature_range=(0, 1))
14 features_scaled = scaler_features.fit_transform(data_features)
15
16 # Also scale the target 'Close' price separately for inverse transformation later
17 scaler_close_dl = MinMaxScaler(feature_range=(0, 1))
18 close_scaled_dl = scaler_close_dl.fit_transform(data[['Close']].values)
19
20 def create_sequences_multi_feature(data_scaled, target_scaled, lookback):
21     X_seq, y_seq = [], []
22     for i in range(lookback, len(data_scaled)):
23         X_seq.append(data_scaled[i-lookback:i, :]) # Use all features for X
24         y_seq.append(target_scaled[i, 0]) # Target remains the Close price
25     return np.array(X_seq), np.array(y_seq)
26
27 # Now, use the multi-feature sequence creation function
28 X_dl, y_dl = create_sequences_multi_feature(features_scaled, close_scaled_dl, LOOKBACK)
29
30 # X_dl is already 3D (samples, lookback, features) due to create_sequences_multi_feature
31 # We need the number of features for the input shape of the models
32 n_features = X_dl.shape[2]
33
34 split_dl = int(len(X_dl) * 0.80)
35 X_tr, X_te = X_dl[:split_dl], X_dl[split_dl:]
36 y_tr, y_te = y_dl[:split_dl], y_dl[split_dl:]
37
38 print(f"Shape of X_tr (multi-feature): {X_tr.shape}")
39 print(f"Number of features (n_features): {n_features}")

```

Shape of X_tr (multi-feature): (1357, 60, 21)
Number of features (n_features): 21

```
1 # -----
2 # 3.2 LSTM Model
3 # -----
4 def build_lstm_multi_feature():
5     model = Sequential([
6         LSTM(128, return_sequences=True, input_shape=(LOOKBACK, n_features)), # Adjusted input_shap
7         Dropout(0.2),
8         LSTM(64, return_sequences=False),
9         Dropout(0.2),
10        Dense(32, activation='relu'),
11        Dense(1)
12    ])
13    model.compile(optimizer=Adam(learning_rate=0.001), loss='mse')
14    return model
15
16 cb = [EarlyStopping(patience=10, restore_best_weights=True),
17       ReduceLROnPlateau(patience=5, factor=0.5)]
18
19 lstm_model = build_lstm_multi_feature()
20 lstm_model.summary()
21 hist_lstm = lstm_model.fit(X_tr, y_tr, epochs=100, batch_size=32,
22                            validation_split=0.1, callbacks=cb, verbose=1)
23
24 y_pred_lstm = scaler_close_d1.inverse_transform(lstm_model.predict(X_te))
25 y_true_d1 = scaler_close_d1.inverse_transform(y_te.reshape(-1,1))
26 results.append(evaluate('LSTM', y_true_d1.flatten(), y_pred_lstm.flatten()))
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 128)	76,800
dropout (Dropout)	(None, 60, 128)	0
lstm_1 (LSTM)	(None, 64)	49,408
dropout_1 (Dropout)	(None, 64)	0
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Total params: 128,321 (501.25 KB)

Trainable params: 128,321 (501.25 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/100

39/39 ----- 17s 283ms/step - loss: 0.0159 - val_loss: 0.0089 - learning_rate: 0.0010

Epoch 2/100

39/39 ----- 13s 101ms/step - loss: 0.0032 - val_loss: 0.0046 - learning_rate: 0.0010

Epoch 3/100

39/39 ----- 4s 106ms/step - loss: 0.0022 - val_loss: 0.0035 - learning_rate: 0.0010

Epoch 4/100

39/39 ----- 5s 131ms/step - loss: 0.0021 - val_loss: 0.0029 - learning_rate: 0.0010

Epoch 5/100

39/39 ----- 4s 103ms/step - loss: 0.0017 - val_loss: 8.8302e-04 - learning_rate: 0.0010

Epoch 6/100

39/39 ----- 4s 112ms/step - loss: 0.0019 - val_loss: 0.0121 - learning_rate: 0.0010

Epoch 7/100

39/39 ----- 5s 120ms/step - loss: 0.0014 - val_loss: 0.0011 - learning_rate: 0.0010

Epoch 8/100

39/39 ----- 4s 100ms/step - loss: 0.0020 - val_loss: 0.0060 - learning_rate: 0.0010

Epoch 9/100

39/39 ----- 5s 115ms/step - loss: 0.0016 - val_loss: 0.0083 - learning_rate: 0.0010

Epoch 10/100

39/39 ----- 5s 117ms/step - loss: 0.0013 - val_loss: 0.0042 - learning_rate: 0.0010

Epoch 11/100

39/39 ----- 4s 102ms/step - loss: 0.0012 - val_loss: 0.0076 - learning_rate: 5.0000e-04

Epoch 12/100

39/39 ----- 5s 127ms/step - loss: 0.0012 - val_loss: 0.0067 - learning_rate: 5.0000e-04

Epoch 13/100

39/39 ----- 4s 104ms/step - loss: 0.0011 - val_loss: 0.0056 - learning_rate: 5.0000e-04

Epoch 14/100

39/39 ----- 5s 101ms/step - loss: 0.0013 - val_loss: 0.0014 - learning_rate: 5.0000e-04

Epoch 15/100

39/39 ----- 6s 165ms/step - loss: 9.8341e-04 - val_loss: 0.0043 - learning_rate: 5.0000e-04

11/11 ----- 2s 110ms/step

LSTM

RMSE : 673.44

MAE : 516.28

MAPE : 2.15%

R2 : 0.6138

```

1 # -----
2 # 3.3 GRU Model
3 # -----
4 def build_gru_multi_feature():
5     model = Sequential([
6         GRU(128, return_sequences=True, input_shape=(LOOKBACK, n_features)), # Adjusted input_shape
7         Dropout(0.2),
8         GRU(64, return_sequences=False),
9         Dropout(0.2),
10        Dense(32, activation='relu'),
11        Dense(1)
12    ])
13    model.compile(optimizer=Adam(learning_rate=0.001), loss='mse')
14    return model
15
16 gru_model = build_gru_multi_feature()
17 hist_gru = gru_model.fit(X_tr, y_tr, epochs=100, batch_size=32,
18                          validation_split=0.1, callbacks=cb, verbose=1)

```

```

19
20 y_pred_gru = scaler_close_dl.inverse_transform(gru_model.predict(X_te))
21 results.append(evaluate('GRU', y_true_dl.flatten(), y_pred_gru.flatten()))

```

```

Epoch 1/100
39/39 ----- 10s 149ms/step - loss: 0.0206 - val_loss: 0.0069 - learning_rate: 0.0010
Epoch 2/100
39/39 ----- 4s 98ms/step - loss: 0.0036 - val_loss: 0.0028 - learning_rate: 0.0010
Epoch 3/100
39/39 ----- 5s 97ms/step - loss: 0.0027 - val_loss: 0.0022 - learning_rate: 0.0010
Epoch 4/100
39/39 ----- 6s 106ms/step - loss: 0.0021 - val_loss: 0.0033 - learning_rate: 0.0010
Epoch 5/100
39/39 ----- 4s 96ms/step - loss: 0.0019 - val_loss: 0.0074 - learning_rate: 0.0010
Epoch 6/100
39/39 ----- 5s 116ms/step - loss: 0.0018 - val_loss: 0.0029 - learning_rate: 5.0000e-04
Epoch 7/100
39/39 ----- 5s 101ms/step - loss: 0.0016 - val_loss: 0.0069 - learning_rate: 5.0000e-04
Epoch 8/100
39/39 ----- 4s 97ms/step - loss: 0.0015 - val_loss: 0.0037 - learning_rate: 5.0000e-04
Epoch 9/100
39/39 ----- 5s 122ms/step - loss: 0.0018 - val_loss: 0.0023 - learning_rate: 5.0000e-04
Epoch 10/100
39/39 ----- 4s 97ms/step - loss: 0.0015 - val_loss: 0.0042 - learning_rate: 5.0000e-04
11/11 ----- 1s 55ms/step

```

```

GRU
RMSE : 1551.57
MAE : 1463.81
MAPE : 5.89%
R2 : -1.0501

```

```

1 #
2 # -----
3 # 3.4 CNN-LSTM Hybrid Model
4 # -----
5 def build_cnn_lstm_multi_feature():
6     model = Sequential([
7         Conv1D(filters=64, kernel_size=3, activation='relu',
8             input_shape=(LOOKBACK, n_features)), # Adjusted input_shape
9         BatchNormalization(),
10        MaxPooling1D(pool_size=2),
11        Conv1D(filters=32, kernel_size=3, activation='relu'),
12        LSTM(64, return_sequences=False),
13        Dropout(0.2),
14        Dense(32, activation='relu'),
15        Dense(1)
16    ])
17    model.compile(optimizer=Adam(learning_rate=0.001), loss='mse')
18    return model
19
20 cnn_lstm_model = build_cnn_lstm_multi_feature()
21 hist_cnn = cnn_lstm_model.fit(X_tr, y_tr, epochs=100, batch_size=32,
22     validation_split=0.1, callbacks=cb, verbose=1)
23
24 y_pred_cnn = scaler_close_dl.inverse_transform(cnn_lstm_model.predict(X_te))
25 results.append(evaluate('CNN-LSTM', y_true_dl.flatten(), y_pred_cnn.flatten()))

```

```

Epoch 1/100
39/39 ----- 5s 39ms/step - loss: 0.0266 - val_loss: 0.1572 - learning_rate: 0.0010
Epoch 2/100
39/39 ----- 1s 37ms/step - loss: 0.0070 - val_loss: 0.1621 - learning_rate: 0.0010
Epoch 3/100
39/39 ----- 1s 29ms/step - loss: 0.0056 - val_loss: 0.1365 - learning_rate: 0.0010
Epoch 4/100
39/39 ----- 1s 22ms/step - loss: 0.0047 - val_loss: 0.0999 - learning_rate: 0.0010
Epoch 5/100
39/39 ----- 1s 22ms/step - loss: 0.0039 - val_loss: 0.1221 - learning_rate: 0.0010
Epoch 6/100
39/39 ----- 1s 23ms/step - loss: 0.0031 - val_loss: 0.0847 - learning_rate: 5.0000e-04
Epoch 7/100
39/39 ----- 1s 23ms/step - loss: 0.0029 - val_loss: 0.0656 - learning_rate: 5.0000e-04
Epoch 8/100
39/39 ----- 1s 22ms/step - loss: 0.0026 - val_loss: 0.0710 - learning_rate: 5.0000e-04

```

```
Epoch 9/100
39/39 ----- 1s 22ms/step - loss: 0.0024 - val_loss: 0.0362 - learning_rate: 5.0000e-04
Epoch 10/100
39/39 ----- 1s 23ms/step - loss: 0.0025 - val_loss: 0.0402 - learning_rate: 5.0000e-04
11/11 ----- 1s 29ms/step

CNN-LSTM
RMSE : 7502.00
MAE : 7456.41
MAPE : 30.24%
R2 : -46.9276
```

```
1 # =====
2 # BLOCK 4: VISUALIZATION
3 # =====
4
5 # 4.1 Plot Actual vs Predicted - LSTM
6 plt.figure(figsize=(14, 6))
7 plt.plot(y_true_dl, label='Actual Nifty 50 Close', color='navy', linewidth=1.5)
8 plt.plot(y_pred_lstm, label='LSTM Predicted', color='red', linewidth=1.5, linestyle='--')
9 plt.plot(y_pred_gru, label='GRU Predicted', color='green', linewidth=1.5, linestyle='-.')
10 plt.plot(y_pred_cnn, label='CNN-LSTM Predicted', color='orange', linewidth=1.5, linestyle=':')
11 plt.title('Nifty 50 - Actual vs Predicted (DL Models)', fontsize=14)
12 plt.xlabel('Trading Days (Test Set)')
13 plt.ylabel('Price (INR)')
14 plt.legend()
15 plt.grid(True, alpha=0.3)
16 plt.tight_layout()
17 plt.savefig('dl_predictions.png', dpi=150)
18 plt.show()
19
20 # 4.2 Model Comparison Bar Chart
21 final_df = pd.DataFrame(results).sort_values('RMSE')
22 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
23 for ax, metric in zip(axes, ['RMSE', 'MAE', 'R2']):
24     colors = ['#2E75B6' if i==0 else '#D3D3D3' for i in range(len(final_df))]
25     ax.bar(final_df['Model'], final_df[metric], color=colors)
26     ax.set_title(f'{metric} Comparison')
27     ax.set_xticklabels(final_df['Model'], rotation=45, ha='right')
28     ax.grid(axis='y', alpha=0.3)
29 plt.suptitle('Model Performance Comparison - Nifty 50 Prediction', fontsize=13)
30 plt.tight_layout()
31 plt.savefig('model_comparison.png', dpi=150)
32 plt.show()
33
34 # 4.3 Training Loss Curves (LSTM)
35 plt.figure(figsize=(10, 4))
36 plt.plot(hist_lstm.history['loss'], label='LSTM Train Loss')
37 plt.plot(hist_lstm.history['val_loss'], label='LSTM Val Loss', linestyle='--')
38 plt.plot(hist_gru.history['loss'], label='GRU Train Loss')
39 plt.plot(hist_gru.history['val_loss'], label='GRU Val Loss', linestyle='--')
40 plt.title('Training & Validation Loss Curves')
41 plt.xlabel('Epoch')
42 plt.ylabel('MSE Loss')
43 plt.legend()
44 plt.grid(alpha=0.3)
45 plt.tight_layout()
46 plt.savefig('loss_curves.png', dpi=150)
47 plt.show()
48
49 # 4.4 Feature Importance (XGBoost)
50 feat_imp = pd.Series(xgb.feature_importances_, index=feature_cols)
51 feat_imp.sort_values().tail(10).plot(kind='barh', figsize=(10,5),
52     color='steelblue', title='Top 10 Feature Importances (XGBoost)')
53 plt.tight_layout()
54 plt.savefig('feature_importance.png', dpi=150)
55 plt.show()
```

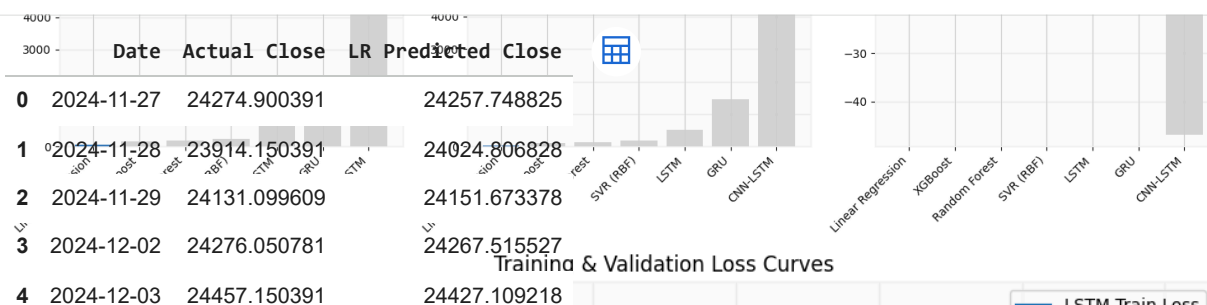

Nifty 50 - Actual vs Predicted (DL Models)

Linear Regression Predictions vs. Actual Prices

```

1 # Get the dates corresponding to the test set for ML models
2 # The 'split' variable is defined where X_train, X_test, y_train, y_test are created.
3 # 'data' DataFrame has the 'Date' column.
4
5 # Ensure 'data' has a proper Date column for slicing
6 if 'Date' not in data.columns:
7     # Assuming 'Date' is the first column after flattening MultiIndex
8     data.rename(columns={'Date': 'Date'}, inplace=True)
9
10 test_dates_ml = data['Date'][split:].reset_index(drop=True)
11
12 # Create a DataFrame to show actual vs. predicted prices for Linear Regression
13 predictions_df_lr = pd.DataFrame({
14     'Date': test_dates_ml,
15     'Actual Close': y_test,
16     'LR Predicted Close': y_pred_lr
17 })
18
19 display(predictions_df_lr.head())

```



```

1 # Plotting Actual vs. Linear Regression Predicted Prices
2 plt.figure(figsize=(14, 7))
3 plt.plot(predictions_df_lr['Date'], predictions_df_lr['Actual Close'], label='Actual Close Price', c
4 plt.plot(predictions_df_lr['Date'], predictions_df_lr['LR Predicted Close'], label='Linear Regression
5 plt.title('Nifty 50 Actual vs. Linear Regression Predicted Close Prices (Test Set)')
6 plt.xlabel('Date')
7 plt.ylabel('Close Price')
8 plt.legend()
9 plt.grid(True)
10 plt.xticks(rotation=45)
11 plt.tight_layout()
12 plt.show()

```

